



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

Experiment: 06

Student Name: Priyanshu Choudhary

Branch: BE CSE

Semester: 6th

Subject Name: System Design

UID: 23BCS12648

Section/Group: KRG_3B

Date of Performance: 25/02/26

Subject Code: 23CSH-314

1. Aim

To design a highly scalable video streaming platform, comparable to YouTube, that effectively supports intensive video uploading, adaptive bitrate streaming, and real-time social interactions while guaranteeing both high availability and exceptionally low latency.

2. Objectives

- To comprehend the overarching architecture of a massive-scale media streaming system.
- To define critical functional requirements, such as direct-to-cloud upload capabilities and personalized content feeds.
- To establish strict non-functional requirements, primarily focusing on CDN caching, immense throughput, and stringent latency constraints.
- To critically analyze the decoupling of synchronous API requests utilizing event-driven stream processing methodologies.
- To engineer robust REST APIs dedicated to video management, content search, and user-driven interactions.

3. Procedure

1. Conducted an in-depth study of real-world video streaming architectures alongside their integrated Content Delivery Networks (CDNs).
2. Defined the fundamental system entities, specifically Users, Videos, Interactions, and View Histories.
3. Investigated the Pre-Signed URL pattern as a strategy to bypass API Gateway bottlenecks during large file uploads.

4. Compiled and categorized the functional and non-functional requirements necessary for seamless media delivery.
5. Formulated API endpoints to handle video processing, retrieve metadata, and facilitate social engagement.
6. Evaluated standard streaming protocols (HLS/DASH) and implemented the separation of thumbnail storage to optimize performance.
7. Analyzed advanced stream processing technologies (Apache Flink/Kafka) to manage real-time, high-volume view aggregation.

4. Functional Requirements

1. The platform must provide secure capabilities for users to register, log in, and manage their personal accounts.
2. The architecture must facilitate the highly efficient uploading of massive raw video files.
3. The system must deliver smooth video playback by utilizing Adaptive Bitrate Streaming, which dynamically adjusts to fluctuating network conditions.
4. The user interface must support real-time social interactions, including functionalities for likes, dislikes, and comments.
5. Users must possess the ability to accurately search for videos utilizing titles or associated metadata.
6. The platform must generate a highly personalized home page feed, curated directly from a user's individualized watch history.

5. Core Entities of the System

- Users
- Videos
- Interactions (Likes/Dislikes)
- Comments
- ViewHistory

6. API Design

1. Authentication API

- a. **Register:** POST /api/v1/auth/register
- b. **Login:** POST /api/v1/auth/login

2. Video & Feed API

- a. **Initialize Upload:** POST /api/v1/videos/upload/init (Returns S3 Pre-Signed URL)
- b. **Get Video Metadata:** GET /api/v1/videos/{video_id}
- c. **Get Personalized Feed:** GET /api/v1/feed
- d. **Search Videos:** GET /api/v1/search?q={query}

3. Interaction API

- a. **Like Video:** POST /api/v1/videos/{video_id}/like
- b. **Add Comment:** POST /api/v1/videos/{video_id}/comments

c. **Get Comments:** GET /api/v1/videos/{video_id}/comments

7. Non-Functional Requirements

1. The infrastructure must be capable of sustaining extremely high throughput to manage concurrent uploads and process millions of simultaneous video views.
2. High Availability must remain a top priority, necessitating an architecture entirely free of any single point of failure.
3. The data architecture will adhere to an Eventual Consistency model for social metrics (such as view counts and likes) to significantly optimize and reduce database write loads.
4. Video delivery latency must be aggressively minimized through the implementation of Edge Caching and a robust Global CDN.
5. The video transcoding pipeline must be designed for horizontal scalability to seamlessly absorb sudden spikes in viral video uploads.

8. High Level Design (HLD)

The overarching system architecture is composed of Client Applications, an API Gateway, and highly specialized microservices dedicated to Authentication, Uploads, Video Metadata, Feeds, and Interactions. To optimize performance, heavy video uploads intentionally bypass the primary backend by utilizing Pre-Signed URLs to securely deposit files directly into Object Storage. Subsequently, asynchronous transcoding pipelines process these raw videos into standardized HLS/DASH segments, which are then distributed and delivered globally via a CDN. Furthermore, high-volume user interactions are efficiently managed employing Redis for Likes, NoSQL clusters for Comments, and a sophisticated Kafka-to-Apache Flink stream processing pipeline to guarantee accurate view aggregation.

9. Outcome

- Successfully architected a highly scalable, fully event-driven video streaming platform.
- Accurately pinpointed the critical functional and non-functional requirements essential for reliable media delivery.
- Constructed comprehensive REST APIs while strategically decoupling asynchronous processing pipelines for maximum efficiency.
- Gained a profound understanding of the practical application of CDNs, adaptive bitrate streaming, and real-time stream processing within massive-scale enterprise systems.